

The session died. The ledger stayed.

On 23 Sep 2026 the Notes work died in the middle of turn 10. The chat window did not keep that turn. The session ledger did.

This note is not a slogan that hooks save every word. It is not a catalog of hook names. It is the mechanism, then the cut we actually have on disk.

The point is small. Each hook writes its line before it returns. A later crash does not delete that line. The same point in simpler words: the record is per event, not per finished answer.

You can object that the last assistant message was empty, so the turn saved nothing. The empty message is a record too. It tells you the answer never reached the wire. The task text was already in the ledger.

What the result depends on

Four parts decide if a cut still has a trail.

1. The agent runtime must send the event to the hook.
2. The hook must append the line to a file before it exits.
3. The file must be outside the chat transcript.
4. A later session must read that file, not the chat.

Drop the chat memory. It is not one of the four parts. The model also does not call tools. It returns a choice. The runtime runs the tool. That split is why two hooks exist.

What must be true

The hook command must be loaded. These two files are the live commands.

- `~/grok/hooks/pwnjournal-logger.py` records what the runtime ran.
- `~/grok/hooks/pwnjournal-model-decisions.py` records what the model chose.

Each command has a timeout of 8 seconds. The shared code is `pwnjournal_common.py`. The workspace root comes from `GROK_WORKSPACE_ROOT`, or from the current directory. On this cut the root was `/home/kali`. The ledger is `/home/kali/logs/sessions/`. It is not the older ledger under `Notes/logs/sessions/`.

The tool hook also copies the same lines into `logs/agent-activity/`. The model hook does not. Its only copy is `model/` inside the session folder.

Both scripts end with exit code 0 after an error. A failed hook must not block the tool. The cost is real. A failed append is silent. You get no line, and you get no error in the tool result.

When this save does not happen

The save is N/A in four cases.

- The event never fired. Text still inside the model is not on the hook bus.
- The append failed, or the process died inside that 8 second hook. That one line is absent.
- You open the wrong ledger. This cut is under `/home/kali/logs/sessions/`, because that was the workspace root.

- You read only the last assistant message. On this cut that message was empty.

The hooks also cannot see private thought tokens. Do not look for them in DECISIONS.md.

How the two hooks differ

Use this split. Mix the two files up and you will invent a run that did not happen.

File	Question it answers
model/DECISIONS.md	What did the model choose?
tools/ACTIONS.md	What did the runtime run, and what came back?

DECISIONS.md is the intent. A PreToolUse line there means the model asked for a tool. ACTIONS.md is the run. A PostToolUse line there means the runtime finished that tool. A choice with no PostToolUse line is a choice that did not finish.

One analogy, then back to the files. This is a lab notebook written at each step. It is not a report written at the end of the day. If the day stops early, the pages already written stay. The conclusion page can be blank.

How a subagent gets a line

A subagent is a second session. The parent does not absorb it. Both sessions write, and they write different facts.

On the parent session. The model chooses `spawn_subagent`. The model hook records that choice as `DELEGATE`, with the child prompt in the tool input. Then `SubagentStart` records that the child is live. If the wire has `lastAssistantMessage`, `SubagentStop` records it with the stop reason.

The parent agent stack is `logs/sessions/.active-agents_<8chars>.json`. `stack_push` runs on `SubagentStart` for the session id on that event. On this cut, that session id was the parent.

On the child session. The child has its own session id. Its first user event is the task prompt, not your chat sentence. Its tool events call `current_agent` on the child id. The parent stack is a different file. The child stack was empty. So the child reads are labeled `main` inside the child folder.

The label `explore` is on the parent `SUBAGENT` lines. It is not on the child `read_file` lines.

The 8 character key. `short()` keeps the first 8 characters of the session id. `ensure_session` maps that short id to one folder. Two children that start in the same time prefix share those 8 characters. They share one folder. The first prompt names the folder. The second prompt does not rename it, because `titled` is already true.

Each JSON line still has `session_full`. You can split the children from that field. The markdown header cannot. It stores one full id.

The clip. A markdown block that is not a final answer keeps 1200 characters (`MAX_SNIP`). A final answer keeps 4000 (`MAX_ASSISTANT`). A user prompt in `tools.jsonl` keeps 6000 (`MAX_PROMPT`). A tool-input field keeps 4000 (`MAX_VAL`).

On this cut both child prompts were shorter than 4000. The JSON lines have the full text. The markdown `DELEGATE` blocks do not. They show a cut of about 1200 characters.

What the cut left on disk

Parent folder:

logs/sessions/2026-09-23_01a0ca9c__create-folder-called-writer-skill-that-mimics-my/

Child folder, both children:

logs/sessions/2026-09-23_01a0cac8__you-are-auditing-whether-a-personal-ideas-skill/

The times are local, from the ledger.

1. 01:51:05. Turn 10 is in the parent DECISIONS.md. The task is the three-pass audit: writing, ideas, then archive the old rules, commands, and agents.
2. 01:51:53. Two DELEGATE lines, both explore. The JSON prompts are 2349 and 2395 characters. Neither JSON prompt is clipped.
3. 01:51:54. Two SUBAGENT lines on the parent. Both say explore is live.
4. 01:51:54. The child folder gets two UserPromptSubmit lines. The full ids end in b8abc3a98e43 and b89c61cfe47e. Same folder. Same short id 01a0cac8.
5. 01:51:57 to 01:51:58. The children read the skill files and the corpus map. They do not read the posts. There is no PostToolUse for a corpus post.
6. 01:52:02. Parent SessionEnd, reason shutdown. The final block says the last assistant message is empty.
7. 01:52:03.132 and 01:52:03.136. Two child SessionEnd lines, both reason shutdown, two different full ids, one folder.

The sign that this trail is enough: a later session read those lines, ran the two audits again, and pushed commit f2bd057. The ledger did not contain the audit answers. The later session had to do that work again.

What stays true when the process dies

Change the condition "the UI process is alive".

What stays true: every event whose hook finished its append. On this cut that set includes the task, both delegate prompts, the files opened, and the reason shutdown.

What is no longer true: any text that never became a hook event. The audit answers are in that set. The children died after the first reads. SubagentStop did not carry a report. An empty final message is the proof of that gap. It is not a proof that the turn was idle.

Change a second condition. Two subagents start in the same 8 character prefix. Separate folders are no longer true. Separate session_full values stay true, because each line stores the full id.

Change a third condition. The prompt is longer than the clip for that file. The full prompt is no longer true in that file. A shorter field in tools.jsonl can still hold it. When the markdown block ends in ... [+N chars], read the JSON line.

How to read a cut session

1. Open logs/sessions/INDEX.md for the workspace root, not for a topic folder.
2. Read model/DECISIONS.md until the last SESSION block. Note the reason.
3. If the final message is empty, read the last GOAL and every DELEGATE block.

4. Open `tools/ACTIONS.md` in the child folder. The last `PostToolUse` line is the last finished step.
5. If two full ids share one folder, split them by `session_full` in `tools.jsonl`.
6. If a markdown prompt ends in `... [+N chars]`, read the `prompt` field in the JSON line.

The limit

For me the takeaway is two lines.

The hook saved the assignment and the first reads, because those events had already appended. It did not save the audit, because that text never reached `Stop` or `SubagentStop`.

Do not treat a shared 8 character folder as two clean sessions. Use `session_full` on each line. The header remembers only the first id.